Gao *et al. Journal of Cloud Computing* (2025) 14:40
https://doi.org/10.1186/s13677-025-00766-5

Journal of Cloud Computing

RESEARCH Open Access

Memory-augment graph transformer based unsupervised detection model for identifying performance anomalies in highly-dynamic cloud environments

Huangyining Gao[1†], Ruyue Xin[2†], Peng Chen[1*], Xi Li[1], Ning Lu[1] and Peng You[1]

Abstract

Cloud computing systems provide highly available and scalable computing, storage, and network resources to meet various service demands. Anomaly detection based on monitoring metrics plays a crucial role in identifying system defects and abnormal behaviors, ensuring the reliability and stability of cloud services. However, as the complexity of data increases and concurrent noise impacts cloud environments, server failures and abnormal events rise, making anomaly detection more challenging. To address these issues, we propose MemGT, an unsupervised multivariate time series anomaly detection method that offers high accuracy, good robustness, and noise resistance for various data patterns in complex cloud environments. Our approach utilizes a Transformer encoder and dynamic graph structure learning to extract spatio-temporal features of monitoring metrics in cloud computing systems in parallel. Additionally, we introduce a novel dynamic gated memory module to guide the Transformer encoder in extracting hidden features, thereby enhancing the model's robustness to varying data patterns in dynamic cloud environments. To accurately distinguish between concurrent noise and real anomalies, we utilize a window-wise graph learning method, further improving the model's noise resistance. We compared the detection performance of MemGT with 15 baseline methods across 8 public datasets. The experimental results demonstrate that our method achieves an average F1 score of 95.04%, surpassing state-of-the-art baseline methods by 24.80%.

Keywords Cloud Computing Systems, Performance Anomalies, Monitoring Metrics, Spatio-temporal Feature Learning, Dynamic Memory Module, Concurrent Noise

†Huangyining Gao and Ruyue Xin contributed equally to this work.

*Correspondence:
Peng Chen
chenpeng@mail.xhu.edu.cn
[1]School of Computer and Software Engineering, Xihua University, Chengdu 610039, China
[2]School of Artificial Intelligence, China University of Geosciences (Beijing), Beijing 100083, China

© The Author(s) 2025. **Open Access** This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by-nc-nd/4.0/.

Introduction

Cloud computing systems consist of a large number of servers, networks, and storage devices that work together to provide highly available and scalable computing and storage services [1]. Accurate detection of performance anomalies, such as high CPU utilization or unexpected workload spikes, can help operators ensure system continuity and maintain service reliability [2, 3]. However, the complex architecture and network topology of cloud infrastructures increase the risk of cascading failures and security vulnerabilities [4]. In addition, the growing interaction between services and workloads changing in cloud applications increases the complexity of cloud computing systems [5, 6]. As a result, anomaly detection in cloud computing systems has become increasingly challenging, driving the demand for robust detection methods with higher accuracy and lower false alarm rates.

In cloud computing systems, monitoring metrics collected from server nodes are often represented as multivariate time series data, which can be analyzed for anomaly detection to deliver fault warnings and maintain system stability [7]. For example, during traffic surges, metrics such as network bandwidth, CPU utilization, and memory usage can help identify performance bottlenecks or resource contention issues. Various methods have been proposed for multivariate time series anomaly detection in cloud computing systems [8]. Traditional machine learning approaches, based on statistical or classification principles, are effective for simple tasks but struggle with the complex nonlinear relationships in cloud environments [9]. Recently, deep learning models, such as recurrent neural networks (RNN) and long short-term memory (LSTM) networks, have shown promise in capturing short-term dependencies and addressing local intricacies [10, 11]. Unlike RNNs and LSTMs, the Transformer model, leveraging a self-attention mechanism, efficiently captures dependencies and models contextual information across multivariate time series, enabling precise anomaly detection in extended sequences [12]. However, these models still struggle to address challenges such as high dimensionality, concurrency, and noise of monitoring metrics in cloud computing systems.

Anomaly detection based on monitoring metrics in cloud environments remains a significant challenge. First, the growing number of monitoring metrics in cloud infrastructures increases data dimensionality, while existing unsupervised methods primarily focus on temporal patterns, often overlooking inter-dimensional correlations. Second, the highly dynamic nature of cloud environments introduces diverse data patterns, yet most detection methods are designed for specific scenarios, resulting in limited accuracy and real-time performance [13, 14]. Moreover, servers and components in cloud systems generate data concurrently, leading to concurrent noise that significantly impacts detection accuracy [15, 16]. Concurrent noise arises from the simultaneous execution of multiple tasks, processes, or data streams in the cloud environment. During periods of unstable network traffic or high system load, multiple dimensions of performance data may be influenced by concurrent noise [15]. As shown in Fig. 1, concurrent noise results in erroneous observations rather than actual anomalies, leading to an increase of the false positive rate in anomaly detection tasks.

To address the above challenges, this paper aims to design a robust and accurate unsupervised anomaly

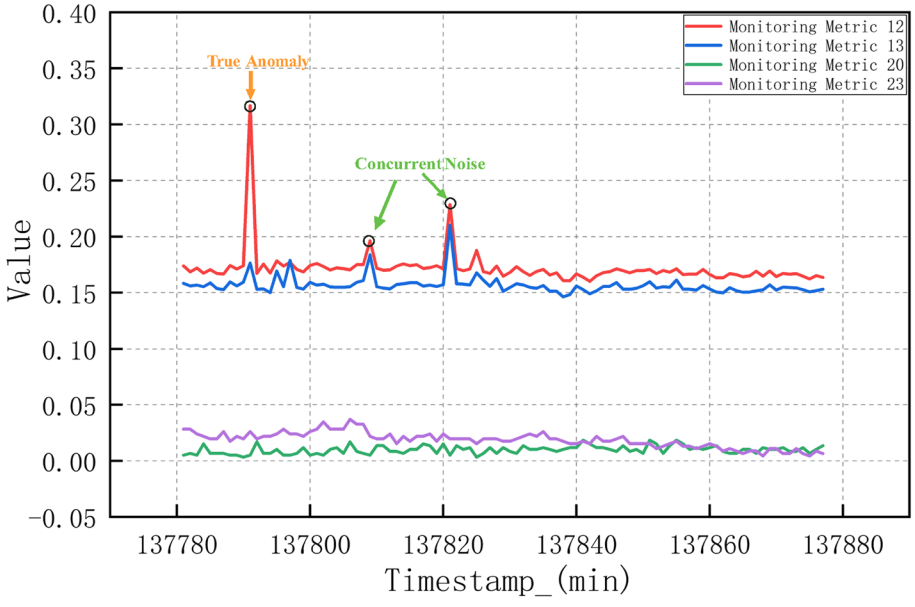


Fig. 1 Concurrent noise vs true performance anomaly in a typical cloud environment

detection model for highly dynamic cloud computing systems with concurrent noise. We mainly focus on the following three research questions:

RQ1: How to accurately extract complex spatio-temporal dependencies from monitoring metrics in cloud environments while meeting the requirements of real-time anomaly detection?
RQ2: How to improve the robustness of anomaly detection models to handle diverse data patterns and adapt to dynamic conditions in complex cloud environments?
RQ3: How to accurately model concurrent noise in cloud environments and enhance the model's resistance to noise?

To answer these questions, we propose MemGT, an unsupervised anomaly detection method that leverages both a Transformer encoder and dynamic graph structure learning in parallel to extract robust spatio-temporal features from monitoring metrics. By interacting with a dynamic gated memory module, MemGT adapts to changes in data patterns, enabling precise extraction of complex features and improving its robustness. Given that concurrent noise stems from disrupted observations rather than actual anomalies, we utilize a novel window-wise Graph Convolutional Network (GCN) to model the concurrent noise, thereby enhancing the model's noise resistance.

The two-stage training framework is adopted for MemGT to addresses two key challenges in highly dynamic cloud environments. Firstly, in unsupervised scenarios, randomly initialized memory units may struggle to capture diverse normal patterns within data, particularly in high-dimensional, noisy cloud environments. Secondly, monitoring metric patterns in cloud environments can dynamically evolve over time, rendering a single static memory insufficient to cover all normal patterns. Thus, in the first stage, we leverage K-means clustering on the query vectors extracted by the Transformer encoder to initialize memory units with representative normal patterns, avoiding the instability and pattern redundancy caused by random initialization. Subsequently, in the second stage, these initialized memory units guide the reconstruction process through an iterative update mechanism where both memory units and query vectors are dynamically adjusted via the gated mechanism. The initialized memory units serve as stable anchors for anomaly discrimination, while their dynamic updating through training helps adapt to evolving data patterns in cloud environments.

The contributions of this paper are as follows:

- We utilize a Transformer encoder and a dynamic graph structure learning approach to extract spatio-temporal features from monitoring metrics, effectively addressing the issues of high-dimensional dependency.
- We design a novel dynamic gated memory module that can introduce new prototype features based on training conditions and iteratively update them, thereby improving the robustness of the model.
- We employ a window-wise GCN to model concurrent noise in data, which significantly enhances the noise resistance ability of the model.
- We provide empirical evaluation for MemGT on eight real-world public datasets, and results show that MemGT achieves state-of-the-art performance compared with other benchmark detection methods.

The remaining sections of this paper are organized as follows: Related work section introduces the related work. Methodology section outlines the overall architecture of the model. Experiment section provides a detailed demonstration of the experiment. Finally, in Conclusion and future work section, we summarize this paper and explore potential future work.

Related work

In this section, we first introduce deep learning-based unsupervised anomaly detection methods, including graph structure learning-based and transformer-based methods. Then, we provide a detailed introduction to the memory network-based anomaly detection methods.

Deep learning-based unsupervised anomaly detection

Deep Learning-based unsupervised deep learning methods have become the mainstream direction of anomaly detection due to their low cost and high adaptability [17]. DAGMM [18] combines Deep Autoencoder (DAE) with Gaussian Mixture Models (GMM) by compressing time series into a latent space and modeling the probability density within this latent space to identify anomalies. OmniAnomaly [11] employs a stochastic RNN to capture the complexity and uncertainty in time series and then utilizes a planar normalization flow to compute reconstruction probabilities for anomaly detection. LSTM-VAE [19] employs LSTM as the fundamental framework to learn temporal patterns within time series data and utilizes Variational AutoEncoder (VAE) to capture efficient data representations. USAD [20] utilizes an encoder-decoder architecture for adversarial training, enabling it to learn how to amplify anomaly reconstruction errors, which improves stability compared to methods based on Generative Adversarial Networks (GAN). MTAD-GAT [21] incorporates parallel feature-directed and time-directed graph attention layers that

can adaptively learn relations between various time series and individual timestamps.

GDN [22] combines structural learning methods and Graph Neural Network(GNN) to capture complex relationships between dimensions and learn normal behavioral patterns of these relationships, thereby detecting data with significant deviations from expected behavior as anomalies. MTGFlow [23] uses a self-attention mechanism to represent the complex relationships between input data as a dynamic and fully connected graph, enabling a more flexible detection of spatial dependencies in time series.TranAD [24] utilizes Transformer networks to capture global temporal dependencies and feature interactions in multivariate time series data, thereby achieving accurate anomaly detection. Anomaly transformer [25] employs a self-attention mechanism to learn the contextual information of input sub-series. It adopts a max-min strategy and calculates the association weight distribution of each time point with the entire sequence to capture complex dynamic dependencies within the time series.

In summary, while these methods utilize deep learning models to achieve unsupervised anomaly detection, the complexity and dynamic nature of data in cloud computing systems often limit their detection accuracy and robustness.

Memory network-based detection methods

Early memory networks predominantly utilized external memory matrices as the primary mechanism to enhance the memory capabilities of neural networks. [26]. Recently, memory networks have been widely applied in the field of anomaly detection. MemAE [27] is the first unsupervised anomaly detection model that combines memory networks with autoencoders and is capable of utilizing normal samples' information to identify anomalies. However, its memory network cannot be updated in real-time during the training process. MNAD [28] proposes a new memory update strategy, which updates the memory network only when normal samples are encountered, preventing it from learning anomaly sample features. However, this method only injects the weighted sum of queries into the memory network without considering the degree of the update. Building upon the previous work, MEMTO [29] introduces a gated memory module, which is capable of adapting to various normal patterns in a data-driven manner. It has achieved excellent results in addressing over-generalization issues and enhancing anomaly detection accuracy.

In summary, the highly dynamic, high-dimensional, and noisy monitoring metrics in cloud environments can limit the accuracy of existing anomaly detection methods. To address these challenges, we introduce a dynamic gated memory module integrated with a Transformer encoder in MemGT to enhance model robustness.

Methodology

We define the monitoring metrics collected from cloud computing systems $X = \{X^1, \ldots, X^N\}$ as a collection of sub-series with L consecutive observations on timestamps, and N represents the total number of sub-series. Each sub-series $X^s = [x_1^s, \ldots, x_L^s] \in R^{L \times d}$ is a sequence of timestamps, where L, d, and $x_t^s \in R^d$ represent the sub-series length, the dimension of the input data, and the observation of the s-th sub-series at time point t, respectively. The task of multivariate time series anomaly detection is to produce the anomaly score of X^s, denoting whether the sub-series is an anomaly [30].

Overall architecture of MemGT

The overall architecture of MemGT is shown in Fig. 2, which includes four parts: I. Memory-augmented Transformer, II. Dynamic Graph Structure Learning, III. Spatio-temporal Feature Fusion, and IV. Concurrent Noise Recognition. Firstly, to extract spatial and temporal features, we input the sub-series X^s collected from cloud environments into the Memory-augmented Transformer and Dynamic Graph Structure Learning modules, respectively. Then, to obtain the comprehensive spatio-temporal information, the output q_t^s from the transformer encoder and the adjacency matrix A_t^s obtained via dynamic graph learning are integrated within a GNN to generate F_t^s. Subsequently, the prototype features $\tilde{q}_t^s$ of normal points from the memory module are concatenated with F_t^s along the feature dimension, generating a robust feature $\tilde{F}_t^s$. Next, this $\tilde{F}_t^s$ is passed into the decoder for reconstruction, and the reconstruction error E_t is subsequently transferred to the Concurrent Noise Recognition module for denoising. Finally, we use the output $\hat{X}^s$ to compute the anomaly score $A(X^s)$.

Memory-augmented transformer

To capture long-term and complex temporal dependencies of monitoring metrics in cloud computing systems, MemGT utilizes the Transformer encoder to project input sub-series into a latent space, as shown in Fig. 2(I). Specifically, given an input sub-series X^s, we first transform it into a fixed-dimensional vector by serving as input to the Transformer's embedding layer. We then utilize the multi-head self-attention mechanism to compute the hidden feature q^s (i.e., the query sequence) that contains rich contextual information. Here, $q^s = [q_1^s, \ldots, q_L^s] \in R^{L \times C}$, where $q_t^s \in R^C (t = 1, \ldots, L)$ is the query vector at time t and C represents the latent dimension.

However, the self-attention mechanism overlooks the positional order of individual time points within a time

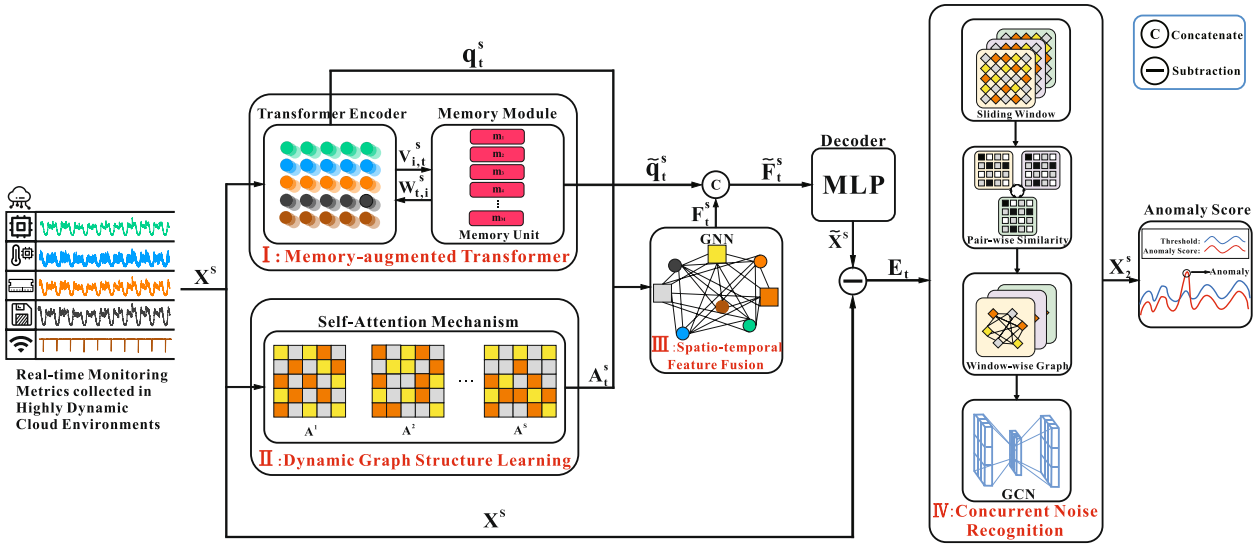


Fig. 2 Overall architecture of MemGT, including I. Memory-augmented Transformer; II. Dynamic Graph Structure Learning; III. Spatio-temporal Feature Fusion; and IV. Concurrent Noise Recognition

series, which may lead to information loss [31]. Although positional encoding has been introduced to address this issue, it lacks transformability in the feature space, potentially resulting in inaccurate representation of positional information in highly dynamic cloud environments [32]. To provide the model with the ability to dynamically adapt to constantly changing data patterns, we design a dynamic gated memory module to guide the process of extracting hidden features from input sub-series. The dynamic gated memory module uses prototype features of normal data stored in the memory units m_i to enhance the encoder's pattern recognition and feature relationship modeling capabilities, where $m_i \in R^C (i = 1, \ldots, M)$ and M denotes the initial number of memory units.

To ensure stable initialization and dynamic adaptation of memory units in highly dynamic cloud environments, MemGT employs a two-stage training. During the first stage, the primary task is to initialize the memory units. Considering the instability risk in the incremental updating process with randomly initialized memory units, MemGT first acquires the query vector q_t^s via the Transformer encoder, then randomly selects 10% as the sample and applies the K-means algorithm to cluster them. The initial values of the memory units are set to the centroids of the clustering. This clustering-based initialization provides geometrically meaningful prototypes that effectively capture core characteristics of normal behavior while maintaining memory sparsity. During the second stage, the memory units m_i and the query vector q_t^s are updated iteratively, regulating the extent to which memory units assimilate new features from current queries, balancing historical information integration with emerging pattern adaptation. We update the memory units by defining the memory attention scores $v_{i,t}^s$ based on the query vector:

$$v_{i,t}^s = P(m_i \to q_t^s) = \frac{exp(\langle m_i, q_i^s \rangle / \tau)}{\sum_{p=1}^{L} exp(\langle m_i, q_p^s \rangle / \tau)}, \quad (1)$$

where τ represents a temperature parameter. To make the training of memory units more flexible, our dynamic gated memory module employs an update gate ψ to control the extent of new normal patterns that memory units should acquire from the query. Additionally, detection tasks in cloud environments typically involve diverse data scales and distributions, making it difficult for a fixed number of memory units to provide dynamic robustness, which may result in potential performance bottlenecks. Therefore, we devise a strategy to dynamically introduce new memory units based on the training progress. Specifically, when the training loss consecutively increases twice, we compute the distances between each pair of current memory units and add a new memory unit at the midpoint of the two units with the largest distance to the memory module, where AN is a hyperparameter denoting the maximum number of new memory units that can be added during training. This approach allows us to maintain a sparse distribution of memory units while dynamically introducing new prototype features. The equation for updating memory units is:

$$\psi = \sigma(U_\psi m_i + W_\psi \sum_{t=1}^{L} v_{i,t}^s q_t^s), \quad (2)$$

$$m_i = (1 - \psi) \odot m_i + \psi \odot \sum_{t=1}^{L} v_{i,t}^s q_t^s, \tag{3}$$

U_ψ and W_ψ are two linear transformations, σ and $\odot$ representing the sigmoid activation function and element-wise multiplication, respectively. Similar to the update of memory units, the update of the query vector begins by defining the query attention scores $w_{i,t}^s$ based on the memory units:

$$w_{t,i}^s = P(q_t^s \to m_i) = \frac{exp(\langle m_i, q_t^s \rangle / \tau)}{\sum_{j=1}^{M} exp(\langle m_j, q_t^s \rangle / \tau)}, \tag{4}$$

where $j = 1, \ldots, M$ is the index for the memory units. We apply $w_{i,t}^s$ as weights and perform a weighted sum of the updated memory units m_i to obtain the updated query vector $\tilde{q}_t^s$:

$$\tilde{q}_t^s = \sum_{i'=1}^{M} w_{t,i'}^s m_{i'}, \tag{5}$$

The memory module enhances the ability to recognize complex feature relationships, enabling the model to adapt to changes in data patterns, and the updated query vector $\tilde{q}_t^s$ better reflects the dynamics of the data. However, for effective feature fusion, we need the global context information of the input sub-series. Therefore, it is better to use the query vector q_t^s as the input to the GNN. As for the updated query vector $\tilde{q}_t^s$, we concatenate it with the spatio-temporal features output by the GNN to obtain new robust features.

Dynamic graph structure learning

The monitoring metrics of cloud computing systems have complex dependencies that dynamically change over timestamps [17, 33]. To extract their spatial features, as illustrated in Fig. 2(II), MemGT uses a self-attention mechanism to learn a dynamic graph from the input data, and treating each individual dimension as a graph node. Given a sub-series X^s, where s represents the index of the current sub-series, the query and key for node i are represented by $X_i^s W^Q$ and $X_i^s W^K$, respectively, $W^Q \in R^{S \times S}$ and $W^K \in R^{S \times S}$ are weights for the query and key. We describe the dependency between feature i and j captured in the s-th sampling as:

$$r_{ij}^s = \frac{(X_i^s W^Q)(X_j^s W^K)^T}{\sqrt{S}}, \tag{6}$$

By calculating the normalized dependencies r_{ij}^s between individual feature i and all other features separately, we can obtain a set of attention scores for feature i:

$$a_{ij}^s = \frac{exp(e_{ij}^s)}{\sum_{j=1}^{K} exp(e_{ij}^s)}, \tag{7}$$

The attention matrix composed of attention scores is used to quantify the pairwise dependencies between features. This attention matrix captures the dynamic relationships among the features of the input data. We then input this spatial information into a GNN as the adjacency matrix A^s.

Spatio-temporal feature fusion

In order to obtain the spatio-temporal features F_t^s of the input data at time t, MemGT uses a GNN to blend the temporal information from the query sequence q_t^s with the dimensional information from graph A_t^s. Meanwhile, we utilize the historical information of the nodes to reduce the error generated in linear transformations and enhance the temporal correlation of the monitoring metrics. The spatio-temporal features F_t^s at time t are defined as:

$$F_t^s = ReLU(A_t^s q_t^s W_1 + q_{t-1}^s W_2) W_3, \tag{8}$$

where W_1 applies a linear transformation on the aggregated spatio-temporal features, W_2 represents the weight of historical information, and W_3 is also a linear transformation that enables the model to capture hierarchical features of the data, leading to enhanced expressiveness and generalization. We input the query vector q_t^s into the gated memory module to train memory units.

We concatenate the spatio-temporal features F_t^s with the updated query vector $\tilde{q}_t^s$ along the feature dimension to obtain the robust spatio-temporal features $\tilde{F}_t^s \in R^{2C}$ for input sub-series. The $\tilde{F}_t^s$ is fed into a decoder for reconstructing the input sub-series. Due to the fact that an overly powerful decoder may result in anomalies and noise being well reconstructed, leading to a decrease in detection accuracy [29]. MemGT employs two fully connected layers as the decoder, and we denote the reconstructed series as $\tilde{X}^s$.

Concurrent noise recognition

Concurrent noise tends to cause large errors simultaneously among multiple features of the input sub-series within a certain time frame [16]. To minimize the impact of concurrent noise on anomaly detection, MemGT applies spatial correlation modeling to the reconstructed monitoring metrics, enabling the filtering out of concurrent noise from identified potential anomalies. Current static GNN approaches can only capture global and stable dependencies between variables, and dynamic GNN methods only predict the evolving dynamic dependencies between variables along the timestamp, making neither suitable for modeling random concurrent noise.

Therefore, we address this problem by using the errors that arise from the reconstruction of the input series to model the graph structure of each sliding window, as illustrated in Fig. 3. Specifically, at time *t*, we utilize the reconstruction error $E_t = \tilde{X}_t^s - X_t^s$ as the embedding for the sliding window, and subsequently calculate the similarity between features *m* and *n* in E_t:

$$sim_t^{mn} = \frac{(E_t^{(m)})^T E_t^{(n)}}{||E_t^{(m)}|| \odot |E_t^{(n)}||}, \tag{9}$$

Naturally, we use the similarity matrix sim_t^{mn} as the adjacency matrix A_t^{mn} for the window GCN at timestamp *t*. Features affected by concurrent noise always exhibit similar fluctuations simultaneously, whereas true anomalies are more likely to show significant deviations on the timestamps. We initially allocate higher weights to features in the graph that simultaneously exhibit similar errors. Features that can be effectively reconstructed by the reconstruction errors of other features with similar errors are likely influenced by concurrent noise, while the opposite is more likely to indicate a true anomaly. Based on this criterion, MemGT employs GCN to perform message passing between variables to reconstruct concurrent noise. The detailed implementation of GCN is shown as follows:

$$\hat{X}^s = \sigma((\tilde{A}X_t^s)W_\theta + b_\theta), \tag{10}$$

where X_t^s is the input sub-series. $\tilde{A} = A - I$, and I represents the identity matrix, σ denotes the activation function, W_θ and b_θ are the learnable weights and bias, respectively. Notably, during the message-passing process, it is necessary to first set elements on the diagonal and above of the adjacency matrix to zero through masking to prevent true anomalies from being effectively reconstructed based on their own information.

Loss function

We define the reconstruction loss L_{rec} as the smooth $L1$ loss between the input sub-series X^s and the reconstructed sub-series $\tilde{X}^s$:

$$L_{rec} = \begin{cases} \frac{1}{2}(X^s - \tilde{X}^s)^2, & |X^s - \tilde{X}^s| < 1 \\ |X^s - \tilde{X}^s)^2| - \frac{1}{2}, & |X^s - \tilde{X}^s| \geq 1 \end{cases} \tag{11}$$

Additionally, we also incorporate two auxiliary losses. The first one is the smooth $L1$ loss between the denoised input sub-series $\hat{X}^s$ and the input sub-series X^s:

$$L_{noi} = \begin{cases} \frac{1}{2}(X^s - \hat{X}^s), & |X^s - \hat{X}^s| < 1 \\ |X^s - \hat{X}^s| - \frac{1}{2}, & |X^s - \hat{X}^s| \geq 1 \end{cases} \tag{12}$$

For the second auxiliary loss, we define the matrix form of the weights $w_{i,t}^s$ as W^s. The probability that the memory units contain information about anomalies grows as the density of W^s increases, which lead to some anomalies being well-reconstructed [29]. To ensure that only a limited number of normal point prototype features reside within the memory units, we introduce an entropy loss L_{entr} to maintain the sparsity of W^s:

$$L_{entr} = \frac{1}{N}\sum_{s=1}^{N}\sum_{t=1}^{L}\sum_{i=1}^{M} -w_{t,i}^s \log(w_{t,i}^s), \tag{13}$$

During the training of MemGT, the objective is to minimize the loss function L. We define L as:

$$L = L_{rec} + \alpha L_{entr} + \beta L_{noi}, \tag{14}$$

where α and β represent the weight coefficients.

Anomaly score

MemGT introduces a criterion that simultaneously considers the input and latent spaces to calculate the anomaly score for each timestamp. Initially, we denote the squared Euclidean distance between the input sub-series and the reconstructed sub-series at time t as $D_1(X_{t_i}^s, \widehat{X}_{t_i}^s)$ to measure the direct deviation at each timestamp:

$$D_1(X_{t_i}^s, \widehat{X}_{t_i}^s) = ||X_{t_i}^s - \widehat{X}_{t_i}^s||_2^2, \tag{15}$$

Since the memory units contain prototype features of normal points, anomalies are typically farther away from

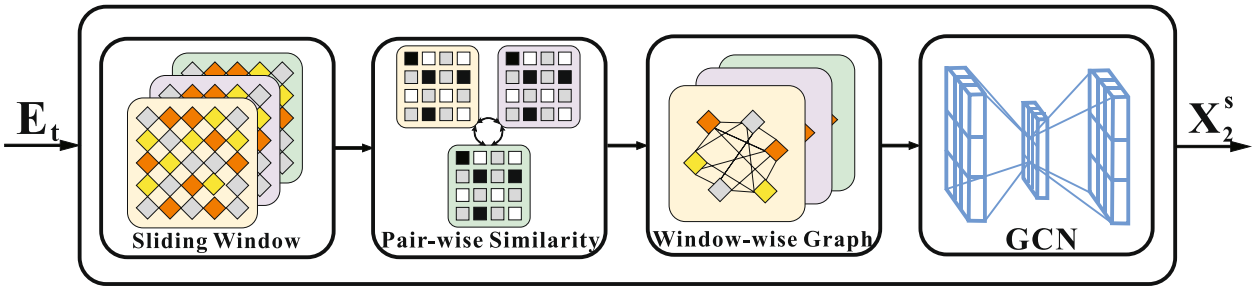


Fig. 3 Generate similarity matrix using sliding window and model the concurrent noise by window-wise GCN

the memory units. Therefore, we use the squared Euclidean distance between the memory units and the spatio-temporal features extracted by the encoder $D_2(F_t^s, m_t^s)$ to calculate the indirect deviation at each timestamp:

$$D_2(F_t^s, m_t^s) = ||(F_t^s - m_t^s)||_2^2, \tag{16}$$

By multiplying the direct deviation degree D_1 with the indirect deviation degree D_2 and then applying softmax, we obtain the final anomaly score as a probability distribution:

$$A(X^s) = softmax(D_1(X_t^s, \widehat{X}_{t,}^s) \odot D_2(F_t^s, m_t^s)), \tag{17}$$

Here, $\odot$ denotes element-wise multiplication. $A(X^s) \in R^L$, and if $A(X^s)$ for a certain timestamp exceeds a threshold, it is classified as an anomaly.

Experiment

In this section, we first describe experimental datasets, baselines, performance metrics, and parameter settings used in the experiments. Then, we conduct five experiments to comprehensively evaluate the performance of MemGT, including:

1) Comparative Experiment and Analysis: Compare MemGT with twelve baseline methods to validate its overall performance.
2) Ablation Study: Verify the effectiveness of each module in MemGT.
3) Parameter Sensitivity Analysis: Evaluate how varying hyperparameter settings affect the performance of detection.
4) Real-time Performance Analysis: Verify the real-time anomaly detection capability of MemGT in cloud environments.
5) Visualization: Verify the effectiveness of MemGT in identifying anomalies through reconstruction errors.

Table 1 presents the comprehensive details of these experiments.

Table 1 Grouping and setting of experiments (A.T represents AnomalyTransformer, I.F represents Isolate Forest)

Experiment	RQ	Methodology	Metric
Ex1 [Comparative experiment and analysis section] Comparative Experiment	RQ1, RQ2, RQ3	LOF, I.F, DAGMM, OmniAnomaly, LSTM-VAE, USAD, MTAD-GAT, GDN, TranAD, A.T, MEMTO, TimesNet, DCdetector, STEN, GDformer.	P R F1
Ex2 [Ablation study section] Ablation Study	RQ1, RQ2, RQ3	Verify the effectiveness of Dynamic Graph Structure Learning, K-means Clustering, Dynamic Gated Memory Module, and Concurrent Noise Recognition.	F1
Ex3 [Parameter sensitivity analysis section] Parameter Sensitivity Analysis	RQ2, RQ3	Verify the effects of window size, number of initial memory units, maximum number of newly added memory units, and coefficient of concurrent noise loss.	F1
Ex4 [Real-time performance analysis section] Real-time Performance Analysis	RQ1	Test the training costs and time needed for performing single-sample anomaly detection on multiple datasets.	Time RAM VRAM
Ex5 [Visualization section] Visualization	RQ1	Visualize the distribution of original series and reconstructed series.	None

Datasets

To rigorously evaluate the effectiveness, robustness, and noise resistance of MemGT under conditions mirroring the core challenges of cloud environments (high dimensionality, high dynamism, concurrent noise), we conduct extensive experiments on eight diverse real-world public datasets:

1) Pooled Server Metrics (PSM) [34] is a dataset collected by eBay that contains monitoring metrics from multiple servers and systems, with 26 features.
2) Secure Water Treatment (SWaT) [35] is collected by iTrust to simulate abnormal situations in water treatment processes and contains 51 features.
3) Water Distribution (WADI) [36] is an expansion and supplement of SWaT, containing 127 features, representing a more complete and realistic water treatment, storage, and distribution network.
4) Mars Science Laboratory (MSL) [37] contains 55 features and is collected by NASA to monitor telemetry data from Mars rovers.
5) Soil Moisture Active Passive (SMAP) [37] includes 25, is collected by NASA to monitor environmental parameters such as soil moisture.
6) The AstrosetHigh/AstrosetLow/AstrosetMiddle datasets [38] are constructed from long-term observations by the Ground-based Wide-Angle Camera Array (GWAC) at the National Astronomical Observatories of the Chinese Academy of Sciences, which monitored stellar brightness as its primary observational target. Within these datasets, anomalous points represent genuine astronomical events (such as flares and outbursts), while noise points correspond to data showing synchronized fluctuations caused by environmental interference (e.g., cloud cover or extreme weather conditions). The datasets were processed following this workflow [16]: First, extract light curves from GWAC's continuously monitored stellar images to form raw multivariate time series (each variable represents a stellar luminosity measurement); next, segment

Table 2 Dataset description (AH, AL, AM represent AstrosetHigh, AstrosetLow, AstrosetMiddle, respectively)

Datasets	Train	Test	Dimensions	Anomalies(%)
PSM	132481	87,841	25	27.8
SWaT	496800	449919	51	11.98
WADI	1048571	172801	127	5.99
MSL	58317	73729	55	10.72
SMAP	135183	427617	25	13.13
AH	8000	6117	38	0.117
AL	6255	2950	40	0.19
AM	5540	5387	54	0.153

time series using sliding windows, preserving typical segments containing astronomical events or environmental noise; finally, establish three signal-to-noise-ratio-characterized subsets: AstrosetHigh, AstroseMiddle, and AstrosetLow by combining astronomical event databases with manual annotation. The Astroset series datasets are available in this link: https://github.com/xinlihao/aero.

While our primary focus is on cloud environments, we include datasets like SWaT, WADI, and the Astroset series for critical evaluation purposes: SWaT and WADI present complex, high-dimensional telemetry data with temporal dependencies and potential anomalies, allowing us to test generalizability on similar structured data (RQ1). Crucially, the Astroset datasets exhibit extremely high levels of concurrent noise and very low anomaly rates, providing an exceptionally challenging testbed to rigorously evaluate the concurrent noise resistance (RQ3) and robustness of MemGT. Besides, these datasets vary in sample sizes, data distributions, data dimensions, noise levels, and anomaly ratios, making them highly suitable for a comprehensive evaluation of MemGT's performance across various anomaly detection tasks in cloud environments. Detail information of these datasets is provided in Table 2 and Fig. 4.

Baselines

We compare MemGT with 15 baseline methods, including 2 classical machine learning based methods (1-2), 10 recently proposed unsupervised detection methods (3-12), and 3 state-of-the-art self-supervised methods (13-15):

1) LOF [39] (ACM SIGMOD 2000): LOF sssdefines the Local Reachability Density (LRD) to measure the neighborhood density of samples, where anomalies are characterized by their density significantly deviating from surrounding neighbors.
2) Isolate Forest [40] (TKDD 2012): Isolation Forest employs a tree structure to recursively and randomly partition data, detecting anomalies by calculating the path length required to isolate samples.
3) DAGMM [18] (ICLR 2018): DAGMM combines a deep autoencoder with a Gaussian mixture model to comprehensively consider low-dimensional feature information and reconstruction errors in the data.
4) OmniAnomaly [11] (KDD 2019): OmniAnomaly integrates Gated Recurrent Unit (GRU) and Variational Autoencoder (VAE) to explicitly model the temporal dependencies and randomness in multivariate time series.
5) LSTM-VAE [19] (ICASSP 2020): LSTM-VAE utilizes the LSTM module to extract long-term trends in time series while combining the VAE module to capture local features.
6) USAD [20] (KDD 2020): USAD integrates an autoencoder architecture with an adversarial training framework, achieving anomaly detection by amplifying the reconstruction error of anomalies.

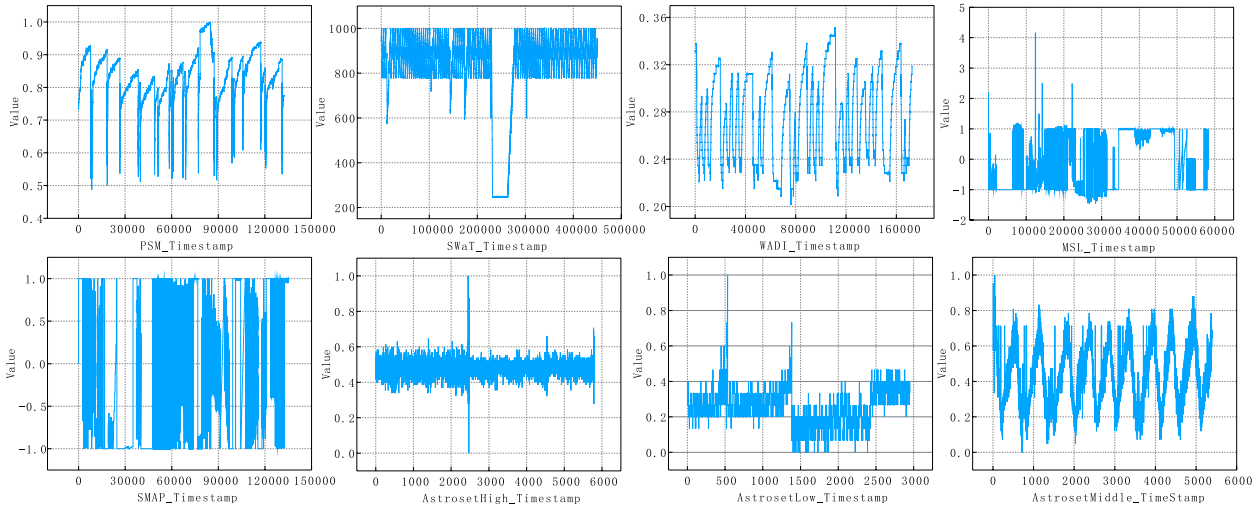


Fig. 4 Typical data distributions in PSM, SWaT, WADI, MSL, SMAP, AstrosetHigh, AstrosetLow, and AstrosetMiddle datasets

7) MTAD-GAT [21] (ICDM 2020): MTAD-GAT utilizes Graph Attention Network (GAT) to capture complex spatial features between time series, and combines GRU decoder to improve the accuracy of anomaly detection.
8) GDN [22] (AAAI 2021): GDN utilizes GNN to capture complex relationships between dimensions, learn normal behavior patterns, detect deviations, and provide anomaly interpretation.
9) TranAD [24] (VLDB 2022): TranAD employs the Transformer framework, leveraging its robust sequence modeling capabilities and self-attention mechanism to capture long-range dependencies for efficient anomaly detection.
10) Anomaly Transformer [25] (ICLR 2022): Anomaly Transformer introduces association discrepancy by simultaneously modeling prior association and sequential association through the Anomaly-Attention mechanism, and utilizes a min-max strategy to amplify these discrepancies.
11) MEMTO [29] (NeurIPS 2023): MEMTO incorporates a gated memory module that is capable of storing prototype features of normal patterns and updating memory items based on input data.
12) GDformer [41] (Arxiv Preprint 2025): GDformer overcomes traditional subsequence isolation limitations by introducing a global dictionary and prototype learning, enabling global representation learning and a unified detection threshold.
13) TimesNet [42] (ICLR 2023): TimesNet captures changes within and between periods by converting one-dimensional time series data into two-dimensional representations, and uses two-dimensional convolution to extract features.
14) DCdetector [43] (KDD 2023): DCdetector effectively distinguishes anomalies from input data by combines dual attention-based contrastive representation learning with the Transformer architecture.
15) STEN [44] (ECML-PKDD 2024): STEN combines Order Prediction-based Temporal Normality (OTN) and Distance Prediction-based Spatial Normality Learning (DSN) modules to capture the spatio-temporal characteristics of time series data.

Performance metrics

We split each dataset into sliding windows of size 100 as input, allocating 80% as the training set and 20% as the validation set. We utilize the most common metrics to evaluate detection performance: precision, recall, and F1 score.

$$precision = \frac{TP}{TP + FP} \tag{18}$$

$$recall = \frac{TP}{TP + FN} \tag{19}$$

$$F1 = 2 \times \frac{precision \times recall}{precision + recall} \tag{20}$$

TP, FP, and FN represent true positive, false positive, and false negative, respectively. Point adjustment is applied within the model to enhance its anomaly detection performance, assuming that if a single time point is identified as anomalous, every other time point within the anomalous segment is also correctly detected.

Parameter settings

We implement our approach using the PyTorch framework on an NVIDIA GTX 4060 Ti 16GB GPU. We generate sub-series by applying a non-overlapped sliding window with a length of 100 to obtain fixed-length inputs for each dataset, and set the batch size for each time step to 256, the number of iterations to 100, and implement an early stop mechanism with the patience of 10. Our model employs the Adam optimizer, with a learning rate of $5e{-}4$ for the first phase of training and $1e{-}4$ for the second phase. We set the initial number of memory units M to 10, the maximum number of newly added memory units AN to 3, and the coefficient of entropy loss α, the coefficient of concurrent noise loss β, and the temperature parameter τ in the softmax function to 0.001, 0.01, and 0.05, respectively.

Comparative experiment and analysis

Comparison results

The detailed comparison results of these models on eight real-world anomaly detection tasks are shown in Table 3. The results show that machine learning-based methods exhibit severe performance degradation under high-dimensional and high-noise conditions, and our MemGT outperforms all other baseline methods on five datasets including PSM, WADI, MSL, SMAP, and AstrosetHigh, improving F1 scores by 9.02%, 35.38%, 9.09%, 8.92%, and 40.8%, respectively. Among these five datasets, MemGT exhibits the most significant improvement on the WADI and AstrosetHigh datasets, where the former has the highest data dimensionality and the latter has the highest noise level. This fully validates the effectiveness of MemGT in high-dimensionality and high-noise scenarios. Crucially, these capabilities——handling high dimensions (numerous sensors/features) and pervasive noise—are also fundamental requirements for anomaly detection in complex IoT systems. MemGT's superior F1 scores across such diverse datasets demonstrate its strong potential for robust performance in IoT applications facing similar data characteristics. In addition, MemGT achieves the second highest F1 score on the SWaT,

Table 3 Detection performance of deep models on eight real-world datasets

Datasets	PSM			SWaT			WADI			MSL		
Methods	**P**	**R**	**F1**	**P**	**R**	**F1**	**P**	**R**	**F1**	**P**	**R**	**F1**
LOF	0.5789	0.9049	0.7061	0.7215	0.6543	0.6862	0.1805	0.4553	0.2584	0.4772	0.8525	0.6118
I.F	0.7609	0.9245	0.8348	0.4929	0.4495	0.4702	0.1192	0.3425	0.1769	0.5394	0.8654	0.6645
DAGMM	0.9349	0.7003	0.8008	0.9933	0.6879	0.8128	0.2348	0.8296	0.3659	0.8230	0.6452	0.7233
OmniAnomaly	0.9198	0.8701	0.8943	0.9782	0.6957	0.8131	0.0760	0.9981	0.4260	0.8065	0.6452	0.7168
LSTM-VAE	0.7362	0.8992	0.8069	0.9624	0.5991	0.7400	0.4587	0.3212	0.3778	0.9009	0.8611	0.7519
USAD	0.9795	0.9646	0.9720	0.9977	0.6879	0.8143	0.2095	0.8296	0.3345	0.7949	1.0000	0.8857
MTAD-GAT	0.3617	0.9646	0.5262	0.9760	0.6957	0.8123	0.2818	0.8012	0.4169	0.6754	1.0000	0.8062
GDN	0.2970	0.9646	0.4542	0.9697	0.6957	0.8101	0.2912	0.7931	0.4260	0.8782	1.0000	0.9351
TranAD	0.8059	0.9646	0.8782	0.9977	0.6879	0.8143	0.3993	0.8296	0.5391	0.7363	1.0000	0.8481
A.T	0.9728	0.9775	0.9751	0.9231	0.9418	0.9320	0.7730	0.6522	0.7044	0.9191	0.9629	0.9405
MEMTO	0.9745	0.9826	0.9785	0.9447	0.9433	0.9439	0.5811	0.4150	0.4397	0.9192	0.9594	0.9388
TimesNet	0.9845	0.9526	0.9672	0.9908	0.7953	0.8824	0.6384	0.4504	0.5280	0.8831	0.7361	0.8029
DCdetector	0.9715	0.9782	0.9748	0.9326	0.9923	**0.9615**	0.7586	0.5927	0.6654	0.9214	0.9600	0.9403
STEN	0.9777	0.9778	0.9777	0.9053	0.9510	0.9276	0.6583	0.9699	0.7843	0.9247	0.8976	0.9109
GDformer	0.9502	0.9882	0.9688	0.9479	0.9375	0.9427	0.8091	0.6419	0.7159	0.9203	0.8326	0.8743
MemGT	0.9742	0.9837	**0.9790**	0.9459	0.9544	0.9500	0.8436	0.8676	**0.8545**	0.9190	0.9641	**0.9410**
Datasets	**SMAP**			**AstrosetHigh**			**AstrosetLow**			**AstrosetMiddle**		
Methods	**P**	**R**	**F1**	**P**	**R**	**F1**	**P**	**R**	**F1**	**P**	**R**	**F1**
LOF	0.5893	0.5633	0.5760	0.1671	0.4349	0.2415	0.3080	0.7771	0.4412	0.3728	1.0000	0.5431
I.F	0.5239	0.5907	0.5553	0.1636	0.3857	0.2297	0.1088	1.0000	0.1963	0.2781	1.0000	0.4351
DAGMM	0.7436	0.7326	0.7380	0.1087	1.0000	0.1960	0.1278	1.0000	0.2556	0.1912	1.0000	0.3210
OmniAnomaly	0.7924	1.0000	0.8842	0.1091	1.0000	0.1967	0.1282	1.0000	0.2564	0.1825	1.0000	0.3087
LSTM-VAE	0.8523	0.7326	0.7879	0.2619	0.2703	0.2660	0.2463	1.0000	0.3953	0.5968	1.0000	0.5968
USAD	0.8139	1.0000	0.8974	0.1086	1.0000	0.1960	0.2766	1.0000	0.4334	0.1960	1.0000	0.3277
MTAD-GAT	0.7518	1.0000	0.8583	0.9434	0.5556	0.6993	0.7496	0.7143	0.7315	0.2409	1.0000	0.3883
GDN	0.8293	1.0000	0.9067	0.1214	1.0000	0.2165	0.2759	1.0000	0.4545	0.6875	1.0000	0.8148
TranAD	0.8155	1.0000	0.8915	0.3810	1.0000	0.5517	0.4484	0.5000	0.4728	0.3294	1.0000	0.4956
A.T	0.9357	0.9930	0.9635	0.9184	1.0000	0.9574	0.9669	1.0000	**0.9831**	0.9504	1.0000	0.9746
MEMTO	0.9360	0.9912	0.9628	0.9221	1.0000	0.9595	0.9677	0.9571	0.9616	0.9528	1.0000	0.9758
TimesNet	0.8958	0.5520	0.6831	0.8012	0.4016	0.5350	0.8654	1.0000	0.9278	0.8919	0.6821	0.7730
DCdetector	0.9351	0.9870	0.9604	0.8980	0.9354	0.9000	0.8259	0.4518	0.5841	0.9466	0.9556	0.9477
STEN	0.9698	0.9509	0.9603	0.9144	1.0000	0.9553	0.9658	0.9286	0.9468	0.9554	1.0000	**0.9772**
GDformer	0.9606	0.8802	0.9186	0.8787	1.0000	0.9354	0.9657	0.6429	0.7719	0.9288	1.0000	0.9631
MemGT	0.9364	0.9927	**0.9637**	0.9240	1.0000	**0.9605**	0.9710	0.9857	0.9781	0.9540	1.0000	0.9765

The best F1 Score in each experiment is in bold, and the second one is underlined. (P, R, I.F, A.T represent Precision, Recall, Isolate Forest and Anomaly Transformer, respectively)

AstrosetLow, and AstrosetMiddle datasets, and the average difference between accuracy and recall in all datasets was only 3.5%, avoiding situations where one metric is extremely high while the other is extremely low. This aids in accurately identifying anomalies while reducing the false alarm rate. In general, MemGT demonstrates excellent performance in handling various anomaly detection tasks in highly dynamic cloud environments.

Performance analysis

The superior performance of MemGT across diverse datasets can be attributed to its unique architectural components effectively addressing the core challenges in cloud anomaly detection. We provide a detailed analysis comparing MemGT with key baselines based on the results in Table 3:

1) Compared to Graph-based Methods (GDN, MTAD-GAT): MemGT consistently outperforms these methods, particularly on complex datasets. On WADI (127 dimension), MemGT achieves an F1 of 0.8545, surpassing GDN by +42.85% (0.8545 vs 0.4260) and MTAD-GAT by +43.76% (0.8545 vs 0.4169). This substantial improvement is due to MemGT's Dynamic Graph Structure Learning (DGTL) module, which learns adjacency matrices per sub-series, adapting to the highly dynamic correlations in cloud metrics, unlike static or globally learned graphs in GDN/MTAD-GAT. Improvement are also clear on AstrosetLow (MemGT 0.9781 vs GDN 0.4545: +52.36%).
2) Compared to Transformer-based Methods (TranAD, Anomaly Transformer): MemGT shows strong improvements over TranAD and Anomaly Transformer where long-term patterns and noise robustness are critical. On WADI, MemGT (0.8545) leads TranAD (0.5391) by +31.54% and Anomaly Transformer (0.7044) by +15.01%. On the noisy AstrosetLow, MemGT (0.9781) outperforms TranAD (0.4728) by +50.53%. This is attributed to MemGT's Dynamic Gated Memory Module (DGMM), which stores/updates normal prototypes, improves reconstruction fidelity for normal points under diverse patterns, and its dedicated Concurrent Noise Recognition (CNR) module. Note: On PSM, MemGT (0.9790) shows a marginal improvement over Anomaly Transformer (0.9751, +0.39%) and TranAD (0.8782, +10.08%), consistent with its design but less dramatic than on noisy datasets.
3) Compared to Memory-based Methods (MEMTO): MemGT generally matches or exceeds MEMTO, with crucial gains in noisy/high-dim scenarios. On WADI, MemGT (0.8545) leads MEMTO (0.4397) by +41.48%. On AstrosetLow, MemGT (0.9781) surpasses MEMTO (0.9571) by +2.1%. The key differentiator is MemGT's CNR module explicitly modeling and filtering concurrent noise via window-wise GCN, which MEMTO lacks. Note: On AstrosetHigh, MEMTO (0.9595) achieves a very slight edge over MemGT (0.9605, –0.10% difference). This minor variance is likely within the margin of experimental stochasticity for this specific dataset, and MemGT maintains superior performance on the more challenging AstrosetLow.
4) Compared to Recent SOTA (DCdetector, STEN, GDformer): MemGT demonstrates clear advantages where dynamic dependencies and noise are prevalent. On WADI, MemGT (0.8545) outperforms DCdetector (0.6654, +18.91%), STEN (0.7843, +7.02%), and GDformer (0.7159, +13.86%). On AstrosetLow, MemGT (0.9781) leads DCdetector (0.5841, +39.40%), STEN (0.9468, +3.13%), and GDformer (0.7719, +20.62%). MemGT's integrated DGSL (dynamic graphs) and CNR (noise filtering) provide this edge over methods relying solely on contrastive learning (DCdetector), static spatio-temporal norms (STEN), or global dictionaries (GDformer). Note: On SWaT, DCdetector achieves a higher F1 (0.9615) than MemGT (0.9500, –1.15% difference). This suggests DCdetector's contrastive learning might be exceptionally effective for SWaT's specific anomaly signatures, while MemGT's strength in noise resistance (CNR) may be less critical here. However, MemGT maintains superior robustness across the broader range of datasets, especially the noisier ones.

In summary, MemGT's leading average performance (95.04% F1, +24.80% over SOTA) is driven by its synergistic design effectively handling cloud-specific challenges:

Table 4 Detection performance of MemGT without key components

Dataset	PSM	SWaT	WADI	MSL
w/o DGSL	0.9786	0.9421	0.6907	0.9381
w/o K-means	**0.9806**	0.9455	0.7966	**0.9421**
w/o DGMM	0.9790	0.9439	0.8510	0.9399
w/o CNR	0.9788	0.9492	0.8443	0.9405
MemGT	0.9790	**0.9500**	**0.8545**	0.9410
Dataset	**SMAP**	**AH**	**AL**	**AM**
w/o DGSL	0.9606	0.9579	0.9651	0.9759
w/o K-means	0.9613	0.9604	0.9458	0.9761
w/o DGMM	**0.9634**	0.9604	0.9625	**0.9765**
w/o CNR	0.9624	0.9596	0.9475	0.9761
MemGT	**0.9634**	**0.9605**	**0.9781**	**0.9765**

The best F1 Score in each experiment is in bold. (DGSL represents the Dynamic Graph Structure Learning, DGMM represents the Dynamic gated Memory Module, CNR represents the Concurrent Noise Recognition, and AH,AL,AM represent AstrosetHigh, AstrosetLow, AstrosetMiddle, respectively)

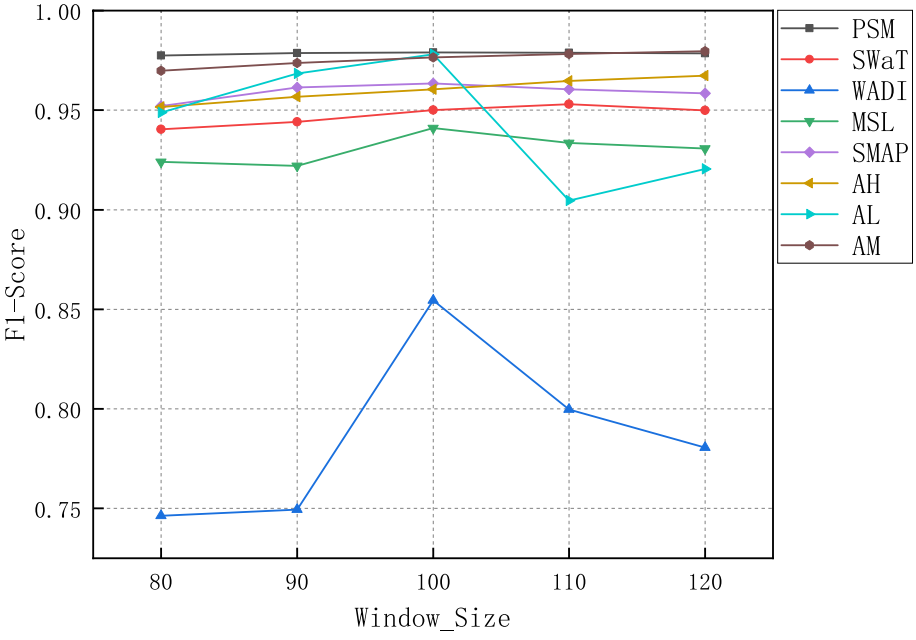


Fig. 5 Performance across different window size

DGSL for dynamic spatial dependencies, DGMM for adaptive normal pattern representation, and crucially, CNR for mitigated concurrent noise. While specific baselines may excel marginally on individual datasets (e.g., DCdetector on SWaT, MEMTO on AstrosetHigh), MemGT consistently delivers high accuracy and robustness across diverse cloud environments, as evidenced by its top performance on 5 of 8 datasets and significant leads on complex, noisy, or high-dimensional datasets like WADI and the Astroset variants.

Ablation study

We then conduct an ablation study of the three key components of MemGT: Dynamic Graph Structure Learning, Dynamic Gated Memory Module, and Concurrent Noise Recognition, to fully demonstrate their effectiveness. The experimental results are shown in Table 4.

Dynamic graph structure learning

The impact of DGSL on detection accuracy is shown in Table 4, w/o DGSL comprehensively reduces the

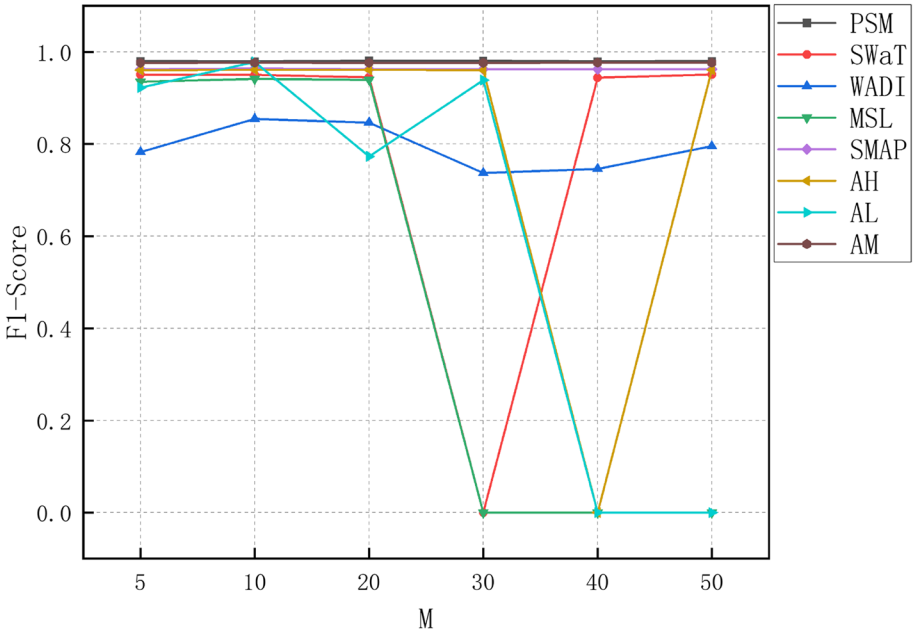


Fig. 6 Performance across different number of initial memory units *M*

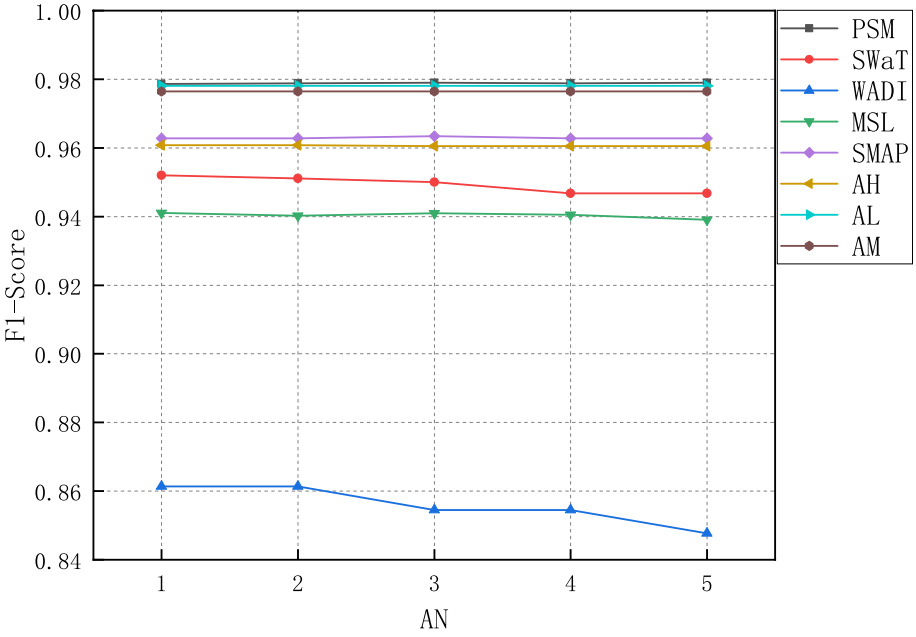


Fig. 7 Performance across different maximum number of newly added memory units *AN*

detection performance of MemGT, especially on the WADI dataset where the F1 score decreased by 16.38%. This fully verifies the effectiveness of the DGSL module in high-dimensional scenarios.

K-means clustering

As shown in Table 4 w/o K-means, we replaced the initialized memory units obtained through K-means clustering with randomly initialized memory units. The results reveal that, except for PSM and MSL, which exhibit higher F1 scores, the performance of all the remaining datasets declines. This suggests that K-means clustering has a positive effect on the detection accuracy of our model in most cases.

Dynamic gated memory module

We verify its effectiveness by removing the DGMM module, as shown in Table 4. Without the DGMM module, the performance of MemGT decreases on most datasets, especially on the SWaT and AstrosetLow datasets, where

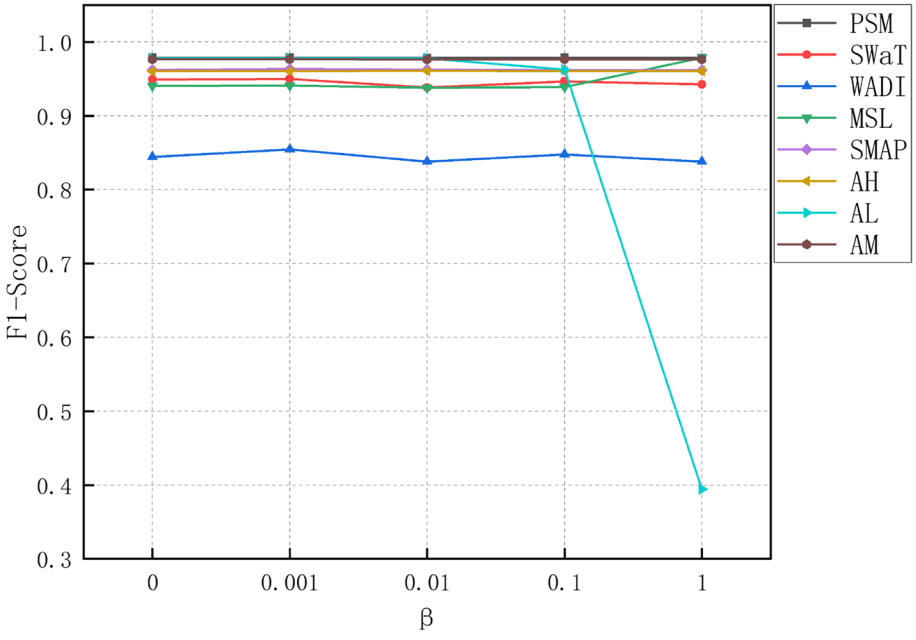


Fig. 8 Performance across different coefficient of concurrent noise loss β

Table 5 The training costs of MemGT compared with three deep learning models on five large-scale datasets, using the metrics Time (seconds), RAM (MB), and VRAM (MB) – the best result for each metric is in bold

Datasets	PSM			SWaT			WADI			MSL			SMAP		
Methods	Time	RAM	VRAM	Time	RAM	VRAM	Time	RAM	VRAM	Time	RAM	VRAM	Time	RAM	VRAM
DCdetector	443	**5157**	9723	8698	8769	17315	1222	5358	10823	347	5211	11980	1038	**4875**	**9684**
A.T	927	5160	10519	3529	7173	10683	471	5376	10525	492	5501	11026	1561	5423	11431
GDformer	347	5158	**2969**	**19**	**7172**	**6279**	203	**5372**	**8740**	178	5213	**7023**	615	5415	11321
MemGT	**165**	8921	16601	373	9588	17323	**59**	5718	11109	**32**	**5177**	9009	**52**	5769	12080

The fastest training time and the minimum RAM/VRAM usage are in bold. (A.T represents Anomaly Transformer)

it results in a 0.61% and 1.56% decrease in F1 scores, respectively. The experimental results indicate that DGMM improves the anomaly detection performance of MemGT.

Concurrent noise recognition

We show the effect of CNR module on the anomaly detection ability of MemGT in Table 4 w/o CNR. The results show that the w/o CNR module has the most significant effect on the noisy AstrosetLow dataset, reducing its F1 score by 3.06%. The CNR module can accurately identify concurrent noise and real anomalies, improve the overall performance of the model, and reduce false alarm rates.

Parameter sensitivity analysis

Window size

Figure 5 illustrates the relationship between MemGT's performance and the window size we used. The results indicate that most datasets are robust to the window size and achieve the highest F1 score when it is set to 100. When the window size is set to other values, the performance of the WADI, AstrosetLow, and MSL datasets declines significantly.

Initial number of memory units M

We investigate the impact of varying the initial number of memory units on our model's detection performance. As shown in Fig. 6, most datasets achieve the highest F1 score when M is set to 10. However, when M exceeds 30, some datasets encounter issues where the F1 score drops to 0 due to training instability, which fully demonstrates the importance of maintaining sparse memory units.

Maximum number of newly added memory units AN

Our memory module is capable of dynamically introducing new memory units based on the training process. To ensure that only a limited number of prototype features are added to the memory module, we analyze the quantity of additional memory units being added. As shown in Fig. 7, both WADI and SWAT achieve the highest F1 scores when the number of AN is 2, and then show a decreasing trend. For the remaining datasets, the best performance is observed when the number of AN is 3. Therefore, we set the number of AN to 3.

Table 6 Testing time of MemGT on eight real-world datasets

Datasets	PSM	SWAT	WADI	MSL
Test Time(μs)	8.27	7.86	1.88	6.43
Datasets	SMAP	AH	AL	AM
Test Time(μs)	4.99	6.52	9.12	7.22

Coefficient of concurrent noise loss β. As shown in Fig. 8, we verify the impact of the value of β on model performance. The larger the value of β, the higher the proportion of concurrent noise loss in the total loss function. The results demonstrate that except for MSL, the other datasets are robust to variations in β. Therefore, we set the value of β to 0.001.

Real-time performance analysis

In this study, we evaluate the real-time performance of MemGT, including training costs and inference speed.

Training costs

In Table 5, we detail the training costs of MemGT on five large-scale datasets, using training time (seconds), Random Access Memory (RAM) consumption (MB), and Video Random Access Memory (VRAM) consumption (MB) as metrics. The results, compared to three deep learning baselines, demonstrate that MemGT exhibits a significant advantage in training speed. It achieves the lowest training time on four out of the five datasets. For example, on the WADI dataset, its training time (59 seconds) is only 12.5% of that required by Anomaly Transformer (471 seconds). Similarly, on the SMAP dataset, it takes only 52 seconds, which is approximately 20 times faster than DCdetector (1038 seconds). However, MemGT's resource consumption is generally higher than other methods. Notably, on the PSM and SWaT datasets, MemGT's VRAM usage is several times higher than that of the most efficient method, GDformer. Nevertheless, this represents a reasonable trade-off resulting from increased model complexity. From a real-time processing perspective, MemGT maintains optimal performance for time-sensitive tasks.

Inference speed

Anomaly detection systems deployed in cloud environments typically require swift identification of anomalies and timely responses. Otherwise, cascading anomalies may occur and rapidly spread, causing more severe damage. Therefore, we record the inference speed of MemGT across eight datasets. The experimental results in Table 6 show that the inference speed of MemGT on a single sample is at the microsecond level, which can meet the real-time detection of anomalies in cloud computing systems. Furthermore, this level of efficiency is critical for real-time monitoring and rapid response in resource-constrained IoT edge environments. In general, MemGT demonstrates excellent performance in handling various anomaly detection tasks in highly dynamic cloud environments and shows promising applicability to IoT scenarios demanding high accuracy, noise resilience, and real-time processing.

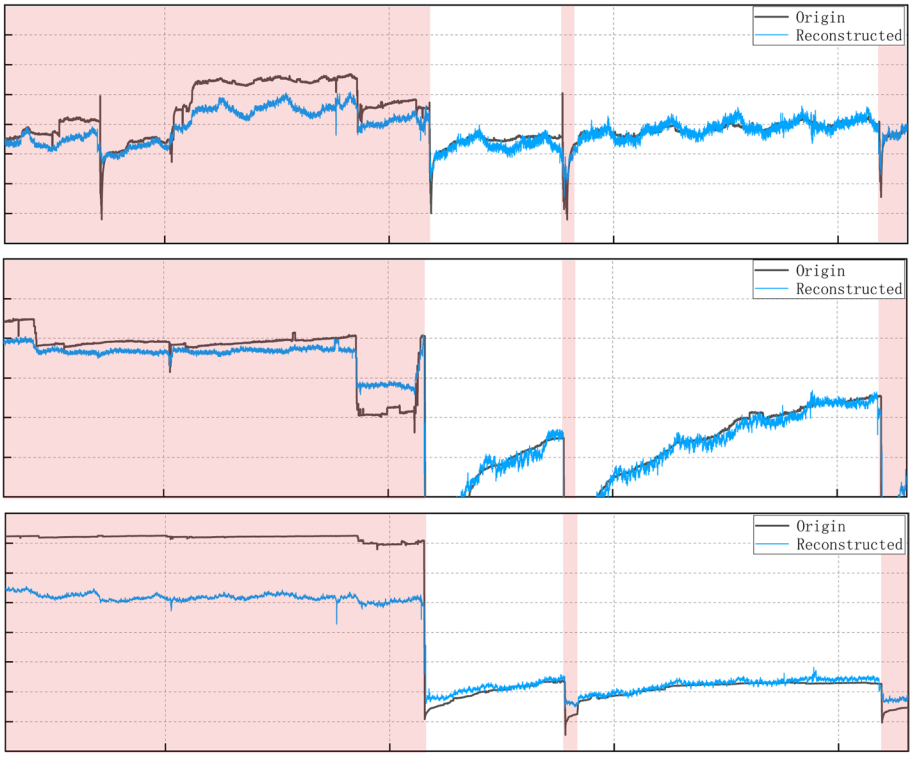


Fig. 9 Visualization of original series and reconstructed series

Visualization

In order to demonstrate that MemGT can effectively identify anomalies through reconstruction errors, we visualize the original and reconstructed sequences, as shown in Fig. 9. We visualize the second, third, and fourth features of the PSM dataset, where the black sequence represents the original sequence and the blue sequence represents the reconstructed sequence. The red shaded area indicates the duration of the abnormal segment. The results show that abnormal segments exhibit significant reconstruction errors, while normal segments can be well reconstructed, fully demonstrating the effectiveness of our architecture.

Conclusion and future work

In cloud computing systems, anomaly detection for performance degradation faces challenges of highly dynamic, high-dimensional, and noisy monitoring metrics. To address these issues, this paper presents MemGT, an unsupervised anomaly detection approach that combines a memory-augmented Transformer with dynamic graph structure learning. MemGT effectively captures intricate spatio-temporal dependencies in input data and effectively tackles the problem of reduced anomaly detection accuracy with high-dimensional datasets. Experiment results indicate that MemGT outperforms other state-of-the-art baselines. Additionally, the window-wise graph learning technique we employ is capable of capturing concurrent noise within the dataset, further enhancing the model's noise resistance.

While the MemGT framework is primarily designed and optimized for detecting performance anomalies in highly dynamic cloud environments, our experimental results on diverse datasets reveal its broader potential. However, it is important to note that while the fundamental techniques show promise, optimal application in non-cloud domains might require domain-specific adaptations. For instance, the physical interpretation and patterns of 'concurrent noise' may differ (e.g., cloud occlusion in astronomy vs. resource contention in clouds), potentially necessitating adjustments to the CNR module's design or input features.

In the future, we will explore explicitly tailoring and evaluating MemGT for specific non-cloud applications, investigated necessary adaptations, and further enhancing its capabilities as a general-purpose robust anomaly detection framework for complex, dynamic, and noisy multivariate time series data.

Authors' contributions

Huangyining Gao: Conceptualization, Methodology, Software, Writing-Original draft preparation. Ruyue Xin: Methodology, Software, Data curation, Writing- Original draft preparation. Peng Chen: Conceptualization, Methodology, Data curation, Supervision, Writing- Reviewing and Editing. Xi Li: Methodology, Software, Data curation. Ning Lu: Methodology, Software, Data curation. Peng You: Methodology, Software, Writing.

Funding

This research is supported by the National Natural Science Foundation of China under Grant No.62376043, Sichuan Provincial Natural Science Foundation under Grant No.2024NSFTD0008, Science and Technology Program of Sichuan Province under Grant No.2024ZHCG0016, Culture and Tourism Program of of Sichuan Province under Grant No.2024YB19. Science and Technology Program of Chengdu under Grant No.2025-GH02-00020-HZ.

Data availability

No datasets were generated or analysed during the current study.

Declarations

Competing interests

The authors declare no competing interests.

Received: 9 March 2025 / Accepted: 27 June 2025
Published online: 22 July 2025

References

1. Qi S, Chen J, Chen P, Wen P, Niu X, Xu L (2024) An efficient gan-based predictive framework for multivariate time series anomaly prediction in cloud data centers. J Supercomput 80(1):1268–1293
2. Xin R, Wang J, Chen P, Zhao Z (2025) Trustworthy ai-based performance diagnosis systems for cloud applications: a review. ACM Comput Surv 57(5):1–37. https://doi.org/10.1145/3701740
3. Islam MS, Pourmajidi W, Zhang L, Steinbacher J, Erwin T, Miranskyy A (2021) Anomaly detection in a large-scale cloud platform. In: 2021 IEEE/ACM 43rd International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP). Madrid, pp 150–159. https://doi.org/10.1109/ICSE-SEIP52600.2021.00024
4. Chen P, Liu H, Xin R, Carval T, Zhao J, Xia Y, Zhao Z (2022) Effectively detecting operational anomalies in large-scale iot data infrastructures by using a gan-based predictive model. Comput J 65(11):2909–2925
5. Song Y, Xin R, Chen P, Zhang R, Chen J, Zhao Z (2024) Autonomous selection of the fault classification models for diagnosing microservice applications. Futur Gener Comput Syst 153:326–339
6. Xin R, Liu H, Chen P, Zhao Z (2023) Robust and accurate performance anomaly detection and prediction for cloud applications: a novel ensemble learning-based framework. J Cloud Comput 12(1):7
7. Xin R, Chen P, Zhao Z (2023) Causalrca: Causal inference based precise fine-grained root cause localization for microservice applications. J Syst Softw 203:111724
8. Soldani J, Brogi A (2022) Anomaly detection and failure root cause analysis in (micro) service-based cloud applications: A survey. ACM Comput Surv (CSUR) 55(3):1–39
9. Ruff L, Kauffmann JR, Vandermeulen RA, Montavon G, Samek W, Kloft M, Dietterich TG, Müller KR (2021) A unifying review of deep and shallow anomaly detection. Proc IEEE 109(5):756–795
10. Malhotra P, Ramakrishnan A, Anand G, Vig L, Agarwal P, Shroff G (2016) Lstm-based encoder-decoder for multi-sensor anomaly detection. arXiv preprint arXiv:1607.00148. https://doi.org/10.48550/arXiv.1607.00148
11. Su Y, Zhao Y, Niu C, Liu R, Sun W, Pei D (2019) Robust anomaly detection for multivariate time series through stochastic recurrent neural network. Proceedings of the 25th ACM SIGKDD international conference on knowledge discovery & data mining, United States. pp 2828–2837. https://doi.org/10.1145/3292500.3330672
12. Vaswani A, Shazeer N, Parmar N, Uszkoreit J, Jones L, Gomez AN, Kaiser Ł, Polosukhin I (2017) Attention is all you need. In: Proceedings of the 31st International Conference on Neural Information Processing Systems (NIPS'17). Curran Associates Inc., Red Hook, pp 6000–6010
13. Liu H, Chen P, Ouyang X, Gao H, Yan B, Grosso P, Zhao Z (2023) Robustness challenges in reinforcement learning based time-critical cloud resource scheduling: A meta-learning based solution. Futur Gener Comput Syst 146:18–33
14. Xin R, Chen P, Grosso P, Zhao Z (2024) A fine-grained robust performance diagnosis framework for run-time cloud applications. Futur Gener Comput Syst 155:300–311

15. Chen P, Xia Y, Pang S, Li J (2015) A probabilistic model for performance analysis of cloud infrastructures. Concurr Comput Pract Experience 27(17):4784–4796
16. Hao X, Chen Y, Yang C, Du Z, Ma C, Wu C, Meng X (2024) From chaos to clarity: time series anomaly detection in astronomical observations. In: 2024 IEEE 40th International Conference on Data Engineering (ICDE). IEEE, Utrecht, pp 570-583. https://doi.org/10.1109/ICDE60146.2024.00050
17. Zhou Q, Chen J, Liu H, He S, Meng W (2023) Detecting multivariate time series anomalies with zero known label. In: Proceedings of the AAAI Conference on Artificial Intelligence, vol 37. Washington, pp 4963–4971. https://doi.org/10.1609/aaai.v37i4.25623
18. Zong B, Song Q, Min MR, Cheng W, Lumezanu C, Cho D, Chen H (2018) Deep autoencoding gaussian mixture model for unsupervised anomaly detection. In: 6th international conference on learning representations, ICLR 2018, Vancouver, April 30–May 3, 2018, conference track proceedings
19. Park D, Hoshi Y, Kemp CC (2018) A multimodal anomaly detector for robot-assisted feeding using an lstm-based variational autoencoder. IEEE Robot Autom Lett 3(3):1544–1551
20. Audibert J, Michiardi P, Guyard F, Marti S, Zuluaga MA (2020) Usad: Unsupervised anomaly detection on multivariate time series. In: Proceedings of the 26th ACM SIGKDD international conference on knowledge discovery & data mining. Washington, pp 3395–3404
21. Zhao H, Wang Y, Duan J, Huang C, Cao D, Tong Y, Xu B, Bai J, Tong J, Zhang Q (2020) Multivariate time-series anomaly detection via graph attention network. In: 2020 IEEE International Conference on Data Mining (ICDM). Sorrrento, pp 841–850. https://doi.org/10.1109/ICDM50108.2020.00093
22. Deng A, Hooi B (2021) Graph neural network-based anomaly detection in multivariate time series. In: Proceedings of the AAAI conference on artificial intelligence, vol 35. pp 4027–4035
23. Dai EY, Chen J (2022) Graph-augmented normalizing flows for anomaly detection of multiple time series. In: Proceedings of the 10th International Conference on Learning Representations
24. Tuli S, Casale G, Jennings NR (2022) Tranad: deep transformer networks for anomaly detection in multivariate time series data. arXiv preprint arXiv:2201.07284. https://doi.org/10.48550/arXiv.2201.07284
25. Xu J, Wu H, Wang J, Long M (2022) Anomaly transformer: time series anomaly detection with association discrepancy. ICLR 2022-10th International Conference on Learning Representations
26. Graves A, Wayne G, Danihelka I (2014) Neural turing machines. arXiv preprint arXiv:1410.5401. https://doi.org/10.48550/arXiv.1410.5401
27. Gong D, Liu L, Le V, Saha B, Mansour MR, Venkatesh S, Hengel Avd (2019) Memorizing normality to detect anomaly: Memory-augmented deep autoencoder for unsupervised anomaly detection. In: Proceedings of the IEEE/CVF international conference on computer vision. Seoul, pp 1705–1714
28. Park H, Noh J, Ham B (2020) Learning memory-guided normality for anomaly detection. In: Proceedings of the IEEE/CVF conference on computer vision and pattern recognition. pp 14372–14381
29. Song J, Kim K, Oh J, Cho S (2024) Memto: memory-guided transformer for multivariate time series anomaly detection. Adv Neural Inf Process Syst 36, New Orleans
30. Chalapathy R, Chawla S (2019) Deep learning for anomaly detection: a survey. arXiv preprint arXiv:1901.03407. https://doi.org/10.48550/arXiv.1901.03407
31. Eldele E, Ragab M, Chen Z, Wu M, Li X (2024) Tslanet: rethinking transformers for time series representation learning. arXiv preprint arXiv:2404.08472. https://doi.org/10.48550/arXiv.2404.08472
32. Kim M, Bhaumik KK, Ali AA, Woo SS (2025) Mixad: memory-induced explainable time series anomaly detection. In: International Conference on Pattern Recognition. Springer, Kolkata, pp 242–257
33. Zheng Y, Koh HY, Jin M, Chi L, Phan KT, Pan S, Chen YPP, Xiang W (2023) Correlation-aware spatial–temporal graph learning for multivariate time-series anomaly detection. IEEE Trans Neural Netw Learn Syst 35(9):11802–11816
34. Abdulaal A, Liu Z, Lancewicki T (2021) Practical approach to asynchronous multivariate time series anomaly detection and localization. Proceedings of the 27th ACM SIGKDD international conference on knowledge discovery & data mining, Singapore, pp 2485–2494
35. Mathur AP, Tippenhauer NO (2016) SWaT: a water treatment testbed for research and training on ICS security. In: International Workshop on Cyber-physical Systems for Smart Water Networks (CySWater). IEEE, USA, pp 31–36
36. Ahmed CM, Palleti VR, Mathur AP (2017) Wadi: a water distribution testbed for research in the design of secure cyber physical systems. In: Proceedings of the 3rd international workshop on cyber-physical systems for smart water networks. Pittsburgh, pp 25–28
37. Hundman K, Constantinou V, Laporte C, Colwell I, Soderstrom T (2018) Detecting spacecraft anomalies using lstms and nonparametric dynamic thresholding. In: Proceedings of the 24th ACM SIGKDD international conference on knowledge discovery & data mining. London, pp 387–395
38. Li GW, Wu C, Zhou GP, Yang C, Li HL, Chen J, Xin LP, Wang J, Haerken H, Ma CH et al (2023) Magnetic activity and parameters of 43 flare stars in the gwac archive. Res Astron Astrophys 23(1):015016
39. Breunig MM, Kriegel H, Ng RT, Sander J (2000) LOF: identifying density-based local outliers. In: Proceedings of the 2000 ACM SIGMOD international conference on management of data, May 16–18, 2000, ACM, Dallas, pp 93–104
40. Liu FT, Ting KM, Zhou ZH (2012) Isolation-based anomaly detection. ACM Trans Knowl Discov Data (TKDD) 6(1):1–39
41. Liu Q, Liu C, Sun S, Yao D, Liang Y (2025) Gdformer: going beyond subsequence isolation for multivariate time series anomaly detection. arXiv preprint arXiv:2501.18196. https://doi.org/10.48550/arXiv.2501.18196
42. Wu H, Hu T, Liu Y, Zhou H, Wang J, Long M (2022) Timesnet: temporal 2d-variation modeling for general time series analysis. In: The eleventh international conference on learning representations. Kigali
43. Yang Y, Zhang C, Zhou T, Wen Q, Sun L (2023) Dcdetector: dual attention contrastive representation learning for time series anomaly detection. In: Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, California, pp 3033–3045
44. Chen Y, Xu H, Pang G, Qiao H, Zhou Y, Shang M (2024) Self-supervised spatial-temporal normality learning for time series anomaly detection. In: Joint European Conference on Machine Learning and Knowledge Discovery in Databases. Springer, Vilnius, pp 145–162

Publisher's Note

Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.